

Design Splitwise

Salesforce · Flipkart · Swiggy · Razorpay · Goldman Sachs · Amazon · Meesho · Groww – LLD rounds lo అత్యధికంగా అడిగే top-5 problems lo ఇది ఒకటి.

ఇది ఒక **data modelling** problem, ఒక algorithm problem కాదు. సరైన ఆకారం ఎంచుకుంటే మొత్తం సమస్య కరిగిపోతుంది. తప్పు ఆకారం ఎంచుకుంటే – మీరు వలయాలని, పేరుకుపోతున్న అప్పులని చేతితో విడదీస్తూ ఉంటారు. ఇక్కడ మనం **తప్పు ఆకారంతో మొదలుపెట్టి**, అది ఎక్కడ విరుగుతుందో చూస్తాం.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output నిజంగా `node lo run చేసినదే` – 3,000-సమూహాల ఒక సరిపోలిక కూడా ఉంది.

ఈ PROBLEM యొక్క ప్రత్యేకత

Parking Lot lo concurrency కష్టం. Cache lo data structure కష్టం. ఇక్కడ రెండూ కాదు – ఇక్కడ కష్టం ఒక్కటే: "నేను దేన్ని store చేయాలి?"

చాలా మంది "అప్పులని" store చేస్తారు, ఎందుకంటే app lo అదే కనిపిస్తుంది. అది తప్పు జవాబు, మరియు §5, §7 lo అది ఎందుకు తప్పు కళ్ళతో చూస్తాం. సరైన జవాబు ఒక్క వాక్యంలో ఉంది, మరియు అది వచ్చాక code చాలా చిన్నదవుతుంది.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు అసలు ప్రశ్న ఏమిటి
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: పేరుకుపోతున్న అప్పులు

3. Step – అప్పులని ఒక జాబితాగా రాయడం
4. మొదటి విరుపు – 19,000 వరుసలు, మరియు రెండు వైపులా అప్పు
5. Step – జతల వారీగా net చేయడం

Part 3 – రెండో విరుపు: వలయం

6. రెండో విరుపు – ముగ్గురూ ₹100 ఇవ్వాలి, నిజానికి ఎవరూ ఇవ్వనవసరం లేదు
7. Step – ఒక్కో వ్యక్తికి ఒకే సంఖ్య

Part 4 – మూడో విరుపు: పరిష్కారం కనిపిస్తుందా?

8. Step – Greedy settlement
9. మూడో విరుపు – greedy ఎప్పుడు ఓడుతుంది
10. ఎంత తరచుగా? – 3,000 సమూహాల మీద కొలత

Part 5 – పంపకం: నాలుగు రకాలు

11. Step – Split ని ఒక మార్చే నియమంగా చేయడం
12. ఫైనల్ సమస్య – ₹100 ని ముగ్గురికి పంచడం

Part 6 – పూర్తి system

13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

Part 7 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

15. నోటీతో చెప్పాల్సిన English script

16. Follow-ups

17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు అసలు ప్రశ్న ఏమిటి

"Design an expense sharing app like Splitwise. A group of friends share expenses; the app tracks who owes whom."

App ని వాడినవాళ్ళకి screen గుర్తుంటుంది: "Ravi owes you ₹450", "You owe Meera ₹200". కాబట్టి సహజమైన ఆలోచన – **ఆ వాళ్ళాలనే store చేద్దాం.**

అదే ఉచ్చు.

అసలు ప్రశ్న ఇది: "Ravi owes you ₹450" అనేది ఒక **వాస్తవమా**, లేక ఒక **లెక్కించిన ఫలితమా?**

జవాబు: అది ఒక **ఫలితం**. నిజమైన వాస్తవాలు **ఖర్చులు** – "Asha ₹8000 కట్టింది, నలుగురం పంచుకున్నాం". "ఎవరు ఎవరికి ఎంత" అనేది వాటి నుంచి వస్తుంది.

ఈ తేడా చిన్నదిగా అనిపిస్తుంది. కానీ మీరు ఫలితాన్ని store చేస్తే – ప్రతి కొత్త ఖర్చుకీ దాన్ని చేతితో సరిచేయాలి, మరియు అక్కడే అన్ని bugs పుడతాయి.

ఒక domain, రెండు model ఎంపికలు

ఎంపిక A – అప్పులని store చేయడం

"ravi → asha ₹450" అనేది ఒక row
కొత్త ఖర్చు = ఉన్న rows ని సరిచేయాలి
వలయాలు మిగిలిపోతాయి (§6)

App lo కనిపించేది ఇదే – అందుకే ఉచ్చు

ఎంపిక B – ఖర్చులని store చేయడం

"asha ₹8000 కట్టింది, 4గురు వంచుకున్నారు"
అప్పులు వీటి నుంచి వస్తాయి
వలయం అనే భావనే ఉండదు

ఒక్కొక్కటి ఒక సంఖ్య చాలు (§7)

ఒక సార్వత్రిక నియమం, ఇక్కడ చిన్నగా కనిపిస్తోంది

జరిగిన సంఘటనలని store చెయ్యి; ప్రస్తుత స్థితిని లెక్కించు.

Bank ledger, git, accounting, event sourcing – అన్నీ ఇదే చేస్తాయి.

స్థితిని store చేస్తే అది క్రమంగా వాస్తవం నుంచి దూరమవుతుంది. సంఘటనలు ఎప్పుడూ అబద్ధం చెప్పవు.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
ఖర్చుని ఎలా పంచాలి – సమానంగానా, వేరే విధంగానా?	సమానం ఒక్కటే అయితే ఒక పంక్తి. లేకపోతే Strategy (§11)
ఒక వ్యక్తి group lo లేకపోయినా ఖర్చులో భాగమవ్వచ్చా?	Validation ఎక్కడ పెట్టాలో ఇది తేలుస్తుంది
"Settle up" అంటే ఏమిటి – ఒక ఖర్చు, లేక వేరేదా?	ఇది ఒక మంచి ప్రశ్న – నిజంగా డబ్బు మారడం ఒక వేరే రకమైన సంఘటన
లావాదేవీల సంఖ్యని కనిష్టం చేయాలా?	అవును అంటే §8-10. ఇది అనుకున్నంత సులభం కాదు
పాత ఖర్చుని edit/delete చేయొచ్చా?	అవును అంటే – balances ని లెక్కించాలి , store చేయకూడదు
ఒకటి కంటే ఎక్కువ currencies?	ఉంటే ప్రతి amount తో ఒక currency, మరియు మార్పిడి రేటు ఎప్పటిది అనే ప్రశ్న
ఫైసలు ఎలా handle చేయాలి?	₹100 ని ముగ్గురికి పంచితే? (§12) – చాలా మంది దీన్ని మర్చిపోతారు

"పాత ఖర్చుని **EDIT** చేయొచ్చా?" – ఈ ప్రశ్న ఒక **DESIGN** ని నిర్ణయిస్తుంది

"లేదు" అయితే – balances ని ఒక running total గా ఉంచొచ్చు.

"అవును" అయితే – ఒక పాత ఖర్చు మారినప్పుడు, దాని ప్రభావాన్ని వెనక్కి తీసి మళ్ళీ వేయాలి. అది సాధ్యమే, కానీ **ఖర్చుల నుంచి balances ని తిరిగి లెక్కించడం** చాలా సులభం మరియు ఎప్పుడూ సరైనది.

నిజ Splitwise lo **edit ఉంది**. కాబట్టి సరైన జవాబు స్పష్టం: **ఖర్చులే సత్యం; balances ఒక derived view**. ఈ doc lo నేను ఒక running total ఉంచుతాను (వేగం కోసం), కానీ ఖర్చుల జాబితా కూడా ఉంచుతాను – ఎప్పుడైనా తిరిగి లెక్కించగలిగేలా.

Part 2 – మొదటి విరుపు: పేరుకుపోతున్న అప్పులు

3. Step – అప్పులని ఒక జాబితాగా రాయడం

App lo కనిపించేదాన్నే model చేద్దాం. ఒక ఖర్చు వచ్చినప్పుడు, ప్రతి పాల్గొనేవాడికీ చెల్లించినవాడికీ మధ్య ఒక అప్పు రాద్దాం:

```
class Splitwise {
  constructor() { this.debts = []; }

  addExpense(payer, amount, participants) {
    const share = amount / participants.length;
    for (const p of participants)
      if (p !== payer) this.debts.push({ from: p, to: payer, amount: share });
  }
}
```

సులభం, స్పష్టం. మూడు ఖర్చులు వేద్దాం – ముగ్గురు స్నేహితులు, ఒక్కొక్కరు ఒక్కోసారి కట్టారు:

```
sw.addExpense('asha', 300, ['asha','ravi','meera']); // భోజనం
sw.addExpense('ravi', 150, ['asha','ravi','meera']); // auto
sw.addExpense('meera', 90, ['asha','ravi','meera']); // టీ
```

మూడు ఖర్చుల తర్వాత అప్పుల జాబితా:

```
ravi → asha ₹100
meera → asha ₹100
asha → ravi ₹50
meera → ravi ₹50
asha → meera ₹30
ravi → meera ₹30
మొత్తం వరుసలు: 6
```

4. మొదటి విరుపు – 19,000 వరుసలు, మరియు రెండు వైపులా అప్పు

ఆ జాబితాలో మొదటి మరియు మూడో వరుసలు చూడండి:

```
ravi → asha ₹100
asha → ravi ₹50
```

Ravi, Asha ₹100 ఇవ్వాలి. Asha, Ravi ₹50 ఇవ్వాలి. ఇది అర్థరహితం – నిజంగా Ravi, Asha ₹50 ఇస్తే చాలు. కానీ మన model రెండింటినీ విడిగా పట్టుకుని కూర్చుంది.

మరియు ఇది పెరుగుతూనే ఉంటుంది:

3 మంది ×	10 ఖర్చులు →	20 వరుసలు
5 మంది ×	50 ఖర్చులు →	200 వరుసలు
10 మంది ×	200 ఖర్చులు →	1800 వరుసలు
20 మంది ×	1000 ఖర్చులు →	19000 వరుసలు

మొదటి విరుపు – $O(\text{ఖర్చులు} \times \text{మంది})$

20 మంది ఒక ఏడాది కలిసి ఉంటే 19,000 అప్పు వరుసలు. మరియు ఈ 19,000 వరుసల్లో ఉన్న సమాచారం నిజానికి **20 సంఖ్యల్లో** పడుతుంది.

కానీ పరిమాణం అసలు సమస్య కాదు. **అసలు సమస్య:** "Ravi నాకు ఎంత ఇవ్వాలి?" అని అడిగితే – మొత్తం జాబితా scan చేసి, రెండు దిక్కులనీ కూడి, తీసివేయాలి. ఆ లెక్క ప్రతిసారీ చేయాలి, మరియు ఎక్కడైనా ఒక వరుస తప్పిపోతే జవాబు తప్పు.

5. Step – జతల వారీగా net చేయడం

స్పష్టమైన మెరుగు: **ప్రతి జతకీ ఒకే వరుస** ఉంచి, అప్పులని అందులోనే కూడటం. `ravi→asha ₹100` మరియు `asha→ravi ₹50` కలిసి `ravi→asha ₹50` అవుతాయి.

```
class PairNet {
  constructor() { this.pairs = new Map(); } // "a|b" → a, b కి ఇవ్వాలిన్నది

  #key(a, b) { return a < b ? `${a}|${b}` : `${b}|${a}`; } // క్రమం స్థిరం
  #sign(a, b) { return a < b ? 1 : -1; } // దిశ ఆ క్రమాన్ని బట్టి

  owe(from, to, amt) {
    const k = this.#key(from, to);
    this.pairs.set(k, (this.pairs.get(k) ?? 0) + amt * this.#sign(from, to));
  }
}
```

Key ని ఎప్పుడూ ఒకే క్రమంలో (`a|b`, alphabetical) ఉంచడం ఒక చిన్న ఉపాయం – లేకపోతే `asha|ravi` మరియు `ravi|asha` రెండు వేర్వేరు entries అవుతాయి, మరియు అవి ఎప్పటికీ కలవవు.

జతల వారీగా net చేశాక:

ravi → asha	₹50
meera → asha	₹70
meera → ravi	₹20

(మొందు 6 వరుసలు ఉండేవి)

6 → 3. బాగా మెరుగైంది. మరియు ఇప్పుడు ఏ జతకీ రెండు దిక్కుల్లో అప్పు లేదు.

ఇది సరైన జవాబే. కాబట్టి ఇక్కడ ఆగిపోవచ్చా?

Part 3 – రెండో విరుపు: వలయం

6. రెండో విరుపు – ముగ్గురూ ₹100 ఇవ్వాలి, నిజానికి ఎవరూ ఇవ్వనవసరం లేదు

ఇప్పుడు ఒక ప్రత్యేకమైన పరిస్థితి పెడదాం. ముగ్గురు స్నేహితులు, ఒక వలయం:

```
c.owe('asha', 'ravi', 100); // Asha, Ravi కి ₹100 ఇవ్వాలి
c.owe('ravi', 'meera', 100); // Ravi, Meera కి ₹100 ఇవ్వాలి
c.owe('meera', 'asha', 100); // Meera, Asha కి ₹100 ఇవ్వాలి
```

జతల వారీ జాబితా:

asha → ravi ₹100

ravi → meera ₹100

meera → asha ₹100

→ ముగ్గురూ ₹100 చెప్పున ఇచ్చివుచ్చుకోవాలి. 3 లావాదేవీలు.

నిజానికి ఎవరికీ ఎవరూ ఏమీ ఇవ్వనవసరం లేదు – అందరి నికర మొత్తం సున్నా.

రెండో విరుపు – జతల వారీ దృష్టి వలయాన్ని చూడలేదు

Asha ₹100 ఇస్తుంది, ₹100 తీసుకుంటుంది. Ravi కూడా. Meera కూడా. ముగ్గురికీ నికర మొత్తం సున్నా. ఎవరూ ఎవరికీ ఏమీ ఇవ్వనవసరం లేదు.

కానీ మన model మూడు లావాదేవీలు కావాలని చెప్తోంది – ఎందుకంటే అది ప్రతి జతనీ విడిగా చూస్తోంది. ఒక జత దృష్టి నుంచి, అవతలివాడు నిజంగానే ₹100 బాకీ ఉన్నాడు.

తప్పు ఎక్కడ? – జత అనేది తప్పు యూనిట్. నిజమైన ప్రశ్న "Asha, Ravi కి ఎంత?" కాదు. నిజమైన ప్రశ్న "Asha కి మొత్తంగా ఎంత రావాలి లేదా ఇవ్వాలి?"

ఇది కేవలం ముగ్గురితో కాదు

మూడు మంది వలయం స్పష్టంగా కనిపిస్తుంది. కానీ 20 మంది group lo, ఆరుగురి గుండా వెళ్ళే ఒక వలయం **ఎవరికీ కనిపించదు**. జతల వారీ model lo అది శాశ్వతంగా అక్కడే ఉండిపోతుంది, మరియు అందరూ అనవసరంగా డబ్బు పంపుకుంటూ ఉంటారు.

7. Step – ఒక్కో వ్యక్తికి ఒకే సంఖ్య

పరిష్కారం ఆశ్చర్యకరంగా చిన్నది. జతలని పూర్తిగా వదిలేసి, **ప్రతి వ్యక్తికీ ఒకే ఒక సంఖ్య** ఉంచుదాం:

నికర నిల్వ (net balance) – ధనాత్మకం అంటే "నాకు ఇంత రావాలి", ఋణాత్మకం అంటే "నేను ఇంత ఇవ్వాలి".

```
class Ledger {
  #bal = new Map();
  #add(p, amt) { this.#bal.set(p, (this.#bal.get(p) ?? 0) + amt); }

  addExpense(payer, amount, ps) {
    const share = amount / ps.length;
    this.#add(payer, amount);           // కట్టాడు → అతనికి రావాలి
    for (const p of ps) this.#add(p, -share); // వాడుకున్నాడు → ఇవ్వాలి
  }
}
```

మొత్తం logic ఐదు పంక్తులు. గమనించండి – payer కి `+amount` కలుపుతున్నాం, ఆపై అతనితో సహా అందరికీ `-share` తీసేస్తున్నాం. Payer కూడా తిన్నాడు కదా.

ఒకే వలయం, మూడు MODELS

asha → ravi ₹100 · ravi → meera ₹100 · meera → asha ₹100

అప్పుల జాబితా

3 వరుసలు
3 లావాదేవీలు కావాలి
వలయం కనిపించదు

జతల వారీ net

3 వరుసలు
3 లావాదేవీలు కావాలి
వలయం ఇంకా కనిపించదు

నికర నిల్వ

asha 0 · ravi 0 · meera 0
0 లావాదేవీలు
వలయం దానంతట అదే కరిగింది

ఇది ఎందుకు పనిచేస్తుంది

ఒక వలయం అంటే – ప్రతి వ్యక్తి ఎంత ఇస్తున్నాడో అంత తీసుకుంటున్నాడు.
నికర నిల్వ ఆ రెండింటినీ ఒకే సంఖ్యలో కూడుతుంది. కాబట్టి అవి **దానంతట అదే రద్దవుతాయి**.
ఇక్కడ ముఖ్యమైనది: మనం వలయాలని **వెలికి తీసి** code రాయలేదు.
సరైన model ఎంచుకున్నాం, మరియు **సమస్య మాయమైంది**. అదే మంచి design యొక్క గుర్తు.

మూడు ఖర్చుల ఉదాహరణ, మరియు వలయం – రెండూ:

నికర నిల్వలు: asha: +120 ravi: -30 meera: -90
లావాదేవీలు : meera→asha ₹90 ravi→asha ₹30

--- అదే వలయం ---

నికర నిల్వలు: (అన్నీ సున్నా)
లావాదేవీలు : (ఏమీ అవసరం లేదు)

6 వరుసలు → 3 → 2. మరియు వలయం **సున్నా** కరిగిపోయింది.

ఒక invariant గమనించండి: అన్ని నికర నిల్వల మొత్తం **ఎప్పుడూ సున్నా**. ఎందుకంటే ప్రతి ఖర్చులో మనం ఎంత కలుపుతామో అంతే తీసేస్తాం. ఇది ఒక అద్భుతమైన **self-check** – మీ balances సున్నాకి కూడకపోతే, ఎక్కడో bug ఉంది. Interview lo ఇది చెప్పడం ఒక బలమైన సంకేతం.

Part 4 – మూడో విరుపు: పరిష్కారం కనిపిస్తున్నామా?

8. Step – Greedy settlement

ఇప్పుడు నికర నిల్వలు ఉన్నాయి: `asha: +120, ravi: -30, meera: -90`. వీటిని నిజమైన లావాదేవీలుగా ఎలా మార్చాలి?

సహజమైన ఆలోచన – అతిపెద్ద అప్పుదారుడిని అతిపెద్ద ఋణదాతతో కలపడం, వీలైనంత తీర్చడం, ఆపై తర్వాతిది:

```
settle() {
  const debtors = this.balances.filter(([,v]) => v < 0)
    .map(([,v]) => [v,-v]).sort((a,b) => b[1]-a[1]);
  const creditors = this.balances.filter(([,v]) => v > 0).sort((a,b) => b[1]-a[1]);

  const tx = []; let i = 0, j = 0;
  while (i < debtors.length && j < creditors.length) {
    const pay = Math.min(debtors[i][1], creditors[j][1]);
    tx.push({ from: debtors[i][0], to: creditors[j][0], amount: +pay.toFixed(2) });
    debtors[i][1] -= pay; creditors[j][1] -= pay;
    if (debtors[i][1] < 0.01) i++;
    if (creditors[j][1] < 0.01) j++;
  }
  return tx;
}
```

ప్రతి అడుగులో **కనీసం ఒకరు పూర్తిగా సరిపోతారు** (ఎందుకంటే `Math.min` తీసుకుంటున్నాం), కాబట్టి ఇది ఎక్కువలో ఎక్కువ **$n-1$ లావాదేవీలు** ఇస్తుంది. అది ఒక మంచి హామీ.

కానీ అది **కనిపిస్తున్నామా?**

9. మూడో విరుపు – greedy ఎప్పుడు ఓడుతుంది

కనిష్టాన్ని కనుక్కోవడానికి ఒక brute-force రాసి, greedy తో పోల్చాను. చాలా సందర్భాల్లో రెండూ సమానం. ఆపై ఇది దొరికింది:

```
counterexample?  balances a:-4 b:-3 c:+2 d:+2 e:+3
                  greedy=4  optimal=3 ← greedy ఎక్కువ!
                  greedy: a→e ₹3  a→c ₹1  b→c ₹1  b→d ₹2
```

మూడో విరుపు - GREEDY ఒక అదనపు లావాదేవీ ఇచ్చింది

Greedy ఏం చేసింది: అతిపెద్ద అప్పుదారుడు $a(-4)$, అతిపెద్ద ఋణదాత $e(+3)$ - కాబట్టి $a \rightarrow e$ ₹3. ఇప్పుడు a కి ₹1 మిగిలింది, అది c కి. ఆపై $b(-3) \rightarrow c$ కి ₹1, d కి ₹2. **మొత్తం 4.**

నిజమైన కనిష్ఠం: $a(-4)$ ని $c(+2)$ మరియు $d(+2)$ తో సరిపోల్చండి - **2 లావాదేవీలు**, మరియు మూడూ పూర్తిగా సున్నా అవుతాయి. ఆపై $b(-3) \rightarrow e(+3)$ - **1 లావాదేవీ. మొత్తం 3.**

Greedy ఓడింది ఎందుకంటే అది **పరిమాణాన్ని** చూసింది, కానీ **ఏ ఉపసమూహాలు సరిగ్గా సున్నాకి కూడతాయో** చూడలేదు.

నిజమైన కనిష్ఠం ఎందుకు కష్టం

"ఖచ్చితంగా సున్నాకి కూడే ఉపసమూహాలని వెతకడం" - ఇది **subset-sum** problem, మరియు అది **NP-hard**. అంటే 20 మంది group కి అన్ని ఉపసమూహాలని పరిశీలించడం ఆచరణలో అసాధ్యం.

Interview lo ఇది చెప్పడం చాలా బలమైనది: "Greedy gives at most $n-1$ transactions, which is a fine guarantee. It isn't always the true minimum though - finding that means looking for subsets that sum to exactly zero, which is subset-sum, so it's NP-hard. For a group of five friends that's irrelevant; I'd ship greedy." - ఇది మీరు పరిమితిని **తెలిసి** ఎంచుకున్నారని చూపిస్తుంది, గమనించలేదని కాదు.

10. ఎంత తరచుగా? - 3,000 సమూహాల మీద కొలత

"అప్పుడప్పుడూ greedy తప్పు" అని చెప్పడం సరిపోదు. **ఎంత తరచుగా, ఎంత తేడాతో?** 3,000 యాదృచ్ఛిక సమూహాలు (4-6 మంది), ప్రతిదానికీ greedy మరియు brute-force కనిష్ఠం:

3000 యాదృచ్ఛిక సమూహాలు (4-6 మంది):
greedy కనిష్ఠమే ఇచ్చింది: 2573 (86%)
greedy ఎక్కువ ఇచ్చింది : 427 (14%) · గరిష్ఠ అదనం 1

కొలత ఒక ఆచరణాత్మక జవాబు ఇచ్చింది

Greedy **86% సందర్భాల్లో కనిష్ఠమే** ఇస్తుంది, మిగతా 14% lo **ఒక్క అదనపు లావాదేవీ** మాత్రమే. ఎప్పుడూ రెండు ఎక్కువ కాదు.

అంటే: ఐదుగురు స్నేహితుల group lo, greedy కి బదులు NP-hard వెతుకులాట చేస్తే మీరు ఆదా చేసేది **ఒక UPI payment**, 14% సందర్భాల్లో. **ఆ మార్పిడి విలువైనది కాదు**, మరియు అది తెలిసి చెప్పడం ఒక engineering నిర్ణయం.

Part 5 – పంపకం: నాలుగు రకాలు

11. Step – Split ని ఒక మారే నియమంగా చేయడం

ఇప్పటిదాకా ప్రతి ఖర్చునీ సమానంగా పంచాం. కానీ నిజ జీవితంలో:

- సమానంగా – హోటల్ గది, నలుగురం
- కచ్చితమైన మొత్తాలు – నేను ₹700, నువ్వు ₹300 (వేరే వేరే వస్తువులు కొన్నాం)
- శాతాల వారీగా – 40/30/20/10
- వాటాల వారీగా – ఒక car లో ఇద్దరు కూర్చున్నారు, ఒక్కరు నడిపాడు → 2:1:1

ఇది ఒక మారే నియమం, మరియు ఇప్పుడు మీకు ఈ కదలిక అలవాటు – **Strategy**:

```
class SplitStrategy {
  shares(amount, participants, config) { throw new Error('subclass implement చేయాలి');
}

class ExactSplit extends SplitStrategy {
  shares(amount, ps, exact) {
    const sum = Object.values(exact).reduce((a, b) => a + b, 0);
    if (Math.abs(sum - amount) > 0.01)
      throw new Error(`SPLIT_MISMATCH: ₹${sum} ≠ ₹${amount}`); // ← తప్పనిసరి తనిఖీ
    return ps.map((p) => [p, exact[p] ?? 0]);
  }
}

class PercentSplit extends SplitStrategy {
  shares(amount, ps, pct) {
    const sum = Object.values(pct).reduce((a, b) => a + b, 0);
    if (Math.abs(sum - 100) > 0.01) throw new Error(`PERCENT_MISMATCH: ${sum}% ≠ 100%`);
    return ps.map((p) => [p, +(amount * (pct[p] ?? 0) / 100).toFixed(2)]);
  }
}

class ShareSplit extends SplitStrategy { // 2:1:1 తరహా
  shares(amount, ps, units) {
    const total = Object.values(units).reduce((a, b) => a + b, 0);
    return ps.map((p) => [p, +(amount * (units[p] ?? 0) / total).toFixed(2)]);
  }
}
```

ఆ validation పంక్తులే అసలు విలువ

ExactSplit మరియు PercentSplit లో ఉన్న తనిఖీలు గమనించండి. ఇవి లేకపోతే – ఎవరో ₹500 ఖర్చుని ₹200 + ₹200 గా పంచితే, **₹100 ఎక్కడికీ వెళ్ళదు**, మరియు నికర నిల్వల మొత్తం ఇక సున్నా కాదు. §7 లో చెప్పిన

invariant నిశ్చబ్దంగా విరిగిపోతుంది.

```
--- తప్పు split ---
SPLIT_MISMATCH: ₹400 ≠ ₹500
PERCENT_MISMATCH: 90% ≠ 100%
```

Interview lo ఈ తనిఖీలని బయటికి చెప్పండి: "Exact and percent splits have to be validated – if the parts don't add up to the whole, the ledger silently stops balancing, and that's the kind of bug nobody notices for months."

12. ఫైసల సమస్య – ₹100 ని ముగ్గురికి పంచడం

₹100 ని ముగ్గురికి సమానంగా పంచండి. ₹33.33 చొప్పున. మొత్తం ₹99.99.

ఒక ఫైసా మాయమైంది. ఒక్క ఖర్చుకీ ఇది పట్టించుకోనవసరం లేదు అనిపిస్తుంది – కానీ ఒక ఏడాదిలో వెయ్యి ఖర్చులు జరిగితే, మీ ledger ₹10 తప్పు అవుతుంది, మరియు **ఎవరికీ ఎందుకో తెలియదు.**

పరిష్కారం రెండు భాగాలు:

1. ఫైసల్స్ లెక్కించండి, రూపాయల్లో కాదు. Floating point ₹ లెక్కలకి పనికిరాదు – `0.1 + 0.2 !== 0.3`. పూర్ణాంకాల ఫైసలు ఎప్పుడూ ఖచ్చితం.

2. మిగిలిన ఫైసలని ఎవరికో ఒకరికి ఇవ్వండి. మాయం చేయొద్దు.

```
class EqualSplit extends SplitStrategy {
  shares(amount, ps) {
    const paise = Math.round(amount * 100);
    const base = Math.floor(paise / ps.length);
    const extra = paise - base * ps.length; // మిగిలిన ఫైసలు
    return ps.map((p, i) => [p, (base + (i < extra ? 1 : 0)) / 100]);
  }
}
```

```
--- ₹100 ని ముగ్గురికి పంచితే ---
[ [ 'a', 33.34 ], [ 'b', 33.33 ], [ 'c', 33.33 ] ]
మొత్తం: 100
```

ఖచ్చితంగా ₹100. మొదటి వ్యక్తి ఒక ఫైసా ఎక్కువ కట్టాడు.

"ఆ అదనపు ఫైసా ఎప్పుడూ మొదటివాడికే ఎందుకు?" – మంచి ప్రశ్న, మరియు interviewer అడగొచ్చు. నిజ systems lo దాన్ని **చెల్లించినవాడికి** ఇస్తారు (అతనికి ఎలాగూ ఎక్కువ వస్తోంది), లేదా ఖర్చుల మధ్య **తిప్పుతారు** (ఈసారి మొదటివాడు, తర్వాతిసారి రెండోవాడు). ముఖ్యమైనది ఏ నియమమో కాదు – **ఒక నియమం ఉండటం**, మరియు మొత్తం ఎప్పుడూ సరిపోవడం.

Part 6 – పూర్తి system

13. మొత్తం code ఒక చోట • నడిపి చూద్దాం

```
class Expense {
  constructor(id, description, payer, amount, participants, strategy, config) {
    Object.assign(this, { id, description, payer, amount, participants });
    this.shares = strategy.shares(amount, participants, config);
    const sum = this.shares.reduce((a, [, v]) => a + v, 0);
    if (Math.abs(sum - amount) > 0.05) // చివరి రక్షణ
      throw new Error(`SHARES_DO_NOT_SUM: ₹${sum} ≠ ₹${amount}`);
    Object.freeze(this); // ఖర్చు ఒక జరిగిన సంఘటన
  }
}

class Group {
  #expenses = []; #bal = new Map();
  constructor(id, members) { Object.assign(this, { id, members }); }
  #add(p, amt) { this.#bal.set(p, +((this.#bal.get(p) ?? 0) + amt).toFixed(2)); }

  addExpense(description, payer, amount, participants, strategy, config) {
    if (!this.members.includes(payer)) throw new Error(`NOT_A_MEMBER: ${payer}`);
    const e = new Expense(`E${this.#expenses.length + 1}`, description,
      payer, amount, participants, strategy, config);
    this.#expenses.push(e); // సంఘటన నిల్వ
    this.#add(payer, amount);
    for (const [p, share] of e.shares) this.#add(p, -share);
    return e;
  }

  settleUp(from, to, amount) { // నిజంగా డబ్బు ఇచ్చినప్పుడు
    this.#add(from, amount); this.#add(to, -amount);
  }

  get balances() {
    return [...this.#bal].filter(([, v]) => Math.abs(v) > 0.01).sort((a, b) => b[1] - a[1]);
  }

  simplify() {
    const debtors = this.balances.filter(([, v]) => v < 0).map(([p, v]) => [p, -v])
      .sort((a, b) => b[1] - a[1]);
    const creditors = this.balances.filter(([, v]) => v > 0).sort((a, b) => b[1] - a[1]);
    const tx = []; let i = 0, j = 0;
    while (i < debtors.length && j < creditors.length) {
      const pay = +Math.min(debtors[i][1], creditors[j][1]).toFixed(2);
      tx.push({ from: debtors[i][0], to: creditors[j][0], amount: pay });
      debtors[i][1] = +(debtors[i][1] - pay).toFixed(2);
      creditors[j][1] = +(creditors[j][1] - pay).toFixed(2);
      if (debtors[i][1] < 0.01) i++;
    }
  }
}
```

```

        if (creditors[j][1] < 0.01) j++;
    }
    return tx;
}

whoOwesWhom(person) {
    const b = this.#bal.get(person) ?? 0;
    return b > 0.01 ? `${person} కి ₹${b.toFixed(2)} రావాలి`
        : b < -0.01 ? `${person} ₹${(-b).toFixed(2)} ఇవ్వాలి`
        : `${person} సరిపోయాడు`;
}
}

```

settleUp ఒక ఖర్చు ఎందుకు కాదు

settleUp ని గమనించండి – అది ఒక Expense సృష్టించట్లేదు, కేవలం రెండు balances ని సరిచేస్తోంది. ఎందుకంటే **నిజంగా డబ్బు మారడం ఒక వేరే రకమైన సంఘటన**. అది ఎవరికీ ఏమీ ఖర్చు చేయలేదు – అది ఒక పాత అప్పుని తీర్చింది.

§2 lo అడిగిన ప్రశ్న ("settle up అంటే ఏమిటి?") కి జవాబు ఇదే, మరియు ఇది code lo నేరుగా కనిపిస్తోంది.

నాలుగు ఖర్చులు, నాలుగు split రకాలు

```

const g = new Group('గోవా ట్రిప్', ['asha','ravi','meera','kiran']);

g.addExpense('హోటల్', 'asha', 8000, ['asha','ravi','meera','kiran'], new
EqualSplit());
g.addExpense('డిన్నర్', 'ravi', 3000, ['asha','ravi','meera','kiran'],
    new PercentSplit(), { ash:40, ravi:30, meera:20, kiran:10 });
g.addExpense('పెట్రోల్', 'meera', 2400, ['ravi','meera','kiran'],
    new ShareSplit(), { ravi:2, meera:1, kiran:1 });
g.addExpense('టికెట్లు', 'kiran', 1000, ['asha','kiran'],
    new ExactSplit(), { ash:700, kiran:300 });

```

నికర నిల్వలు:

```

asha   +₹4100
meera  ₹-800
ravi   ₹-1100
kiran  ₹-2200

```

పరిష్కారం (3 లావాదేవీలు):

```

kiran → ash   ₹2200
ravi  → ash   ₹1100
meera → ash   ₹800

```

ravi ₹1100.00 ఇవ్వాలి

ఈ output ని తనిఖీ చేద్దాం

Asha యొక్క +4100 ని చేతితో లెక్కిద్దాం:

ఖర్చు	Asha కట్టింది	Asha వాటా	నికరం
హోటల్ ₹8000, 4గురు సమానం	+8000	-2000	+6000
డిన్నర్ ₹3000, 40%	0	-1200	-1200
పెట్రోల్ ₹2400	0	0 (పాల్గొనలేదు)	0
టికెట్లు ₹1000, exact ₹700	0	-700	-700
		మొత్తం	+4100 ✓

మరియు అన్ని నిల్వల మొత్తం: $4100 - 800 - 1100 - 2200 = 0 \checkmark$ – §7 lo చెప్పిన invariant నిజం.

మూడో ఖర్చు గమనించండి – Asha అందులో పాల్గొనలేదు (['ravi', 'meera', 'kiran'] మాత్రమే). ఆ ఖర్చు ఆమె నిల్వని అస్సలు తాకలేదు. ఇది సహజంగా పనిచేసింది ఎందుకంటే మనం participants జాబితాకే share ఇస్తున్నాం, group మొత్తానికి కాదు.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	ఖర్చు, పాల్గొనేవాళ్ళు	మౌలిక భావనలు
§4 (విరుపు)	–	అప్పుల జాబితా $O(\text{ఖర్చులు} \times \text{మంది})$
§6 (విరుపు)	–	జతల వారే దృష్టి వలయాలని చూడలేదు
§7	<code>#bal</code> – ఒక్కో వ్యక్తికి ఒక సంఖ్య	వలయం దానంతట అదే కరిగింది
§8	<code>simplify()</code> greedy	నిల్వల నుంచి లావాదేవీలు
§11	<code>SplitStrategy</code> + validation	పంపకం ఒక మారే నియమం
§12	ఫైసల లెక్క	₹0.01 మాయం కాకూడదు
ఇక్కడ	<code>#expenses</code> , <code>settleUp</code> , <code>Object.freeze</code>	సంఘటనలే సత్యం (§1)

Part 7 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

ఈ problem Io code తక్కువ, ఆలోచన ఎక్కువ – సమయాన్ని అలాగే వంచండి

5 నిమి

Clarify – split రకాలు, edit, కనిష్ట లావాదేవీలు

12 నిమి – Modelling

అప్పులు → జతలు → నికర నిల్వ · ఇదే కేంద్రం

8 నిమి

Split strategies + validation

10 నిమి

Greedy settlement + అది కనిష్టం కాదని

10 నిమి

ఫైసలు, follow-ups, edge cases

ఏమి తప్పక చెప్పాలి

1. సంఘటనలు vs స్థితి (§1) – "expenses are the truth, balances are derived." ఇది మీ మొదటి వాక్యం కావచ్చు.
2. వలయం (§6) – మీరే ఒక 3-వ్యక్తుల వలయాన్ని గీసి, జతల వారే model ఎలా విఫలమవుతుందో చూపించండి.
3. Greedy కనిష్టం కాదు, మరియు నిజమైన కనిష్టం NP-hard (§9) – ఇది చాలా తక్కువ మంది చెప్తారు.
4. మొత్తం సున్నా invariant (§7) – ఒక self-check ఉందని చూపించడం.
5. ఫైసల సమస్య (§12) – ₹100 ÷ 3.

ఏమి వదిలేయాలి

- నాలుగు split strategies రాయొద్దు – EqualSplit మరియు ExactSplit చాలు; మిగతావి "అదే నమూనా".
- అప్పుల జాబితా version ని రాయొద్దు – దాన్ని ఒక వాక్యంలో చెప్పి, ఎందుకు తప్పు వివరించండి.
- whoOwesWhom వదిలేయండి – అది స్పష్టం.

15. నీటితో చెప్పాల్సిన English script

"Let me start with what I'd store, because I think that's the whole problem here.

The obvious move is to store debts – 'Ravi owes Asha 450' – because that's what the app shows. I don't think that's right. That sentence is a **derived result**, not a fact. The facts are the expenses: who paid, how much, and who shared it. If I store the derived version, every new expense means patching existing rows, and that's where the bugs live. It also makes editing an old expense very painful, and real Splitwise lets you edit.

Even if I net each pair, there's a case that breaks: three people in a cycle, each owing the next a hundred. Pairwise, that's three debts and three transfers. But every one of them is net zero – nobody needs to pay anybody. A pair-based view can't see that, and in a group of twenty a cycle through six people is invisible.

So the right unit isn't the pair, it's the person. One net balance each: positive means you're owed, negative means you owe. Adding an expense credits the payer the full amount and debits everyone their share, payer included. Five lines, and the cycle just dissolves – I never wrote cycle-detection code, the model made the problem disappear. There's a nice invariant too: all balances always sum to zero, which is a free self-check.

To turn balances into actual transfers I'd match the largest debtor to the largest creditor greedily. That gives at most $n-1$ transactions. I should be honest though – it's not always the true minimum. I tested it: with balances of minus four, minus three, plus two, plus two and plus three, greedy needs four transfers but three is possible, because minus four pairs exactly with the two plus-tuos. Finding those exact-sum subsets is subset-sum, so the true minimum is NP-hard. I measured greedy against brute force over three thousand random groups – it was optimal 86 percent of the time and never off by more than one transfer. For a group of friends, I'd ship greedy.

Splitting itself is a strategy – equal, exact, percentage, shares. The exact and percentage ones have to validate that the parts sum to the whole, otherwise the ledger silently stops balancing.

Last thing: money. A hundred rupees split three ways is 33.33 each, which is 99.99 – a paisa vanishes, and over a thousand expenses that's real drift. I'd compute in integer paise and give the remainder to someone deterministically rather than letting it disappear."

16. Follow-ups

Follow-up	జవాబు	మారే classes
"కొత్త split రకం (ఉదా: adjustment)"	కొత్త <code>SplitStrategy</code>	+1 కొత్తది, 0 edits
"పాత ఖర్చుని edit/delete"	<code>#expenses</code> నుంచి balances తిరిగి లెక్కించడం	ఒక <code>recompute()</code>
"బహుళ currencies"	Amount తో ఒక currency; balance ఒక్కో currency కి ఒకటి	<code>#bal</code> ఒక nested Map
"ఖర్చుకి ఒక ఫోటో/గమనిక"	<code>Expense</code> కి fields	0 concepts
"Group కాకుండా ఇద్దరి మధ్య"	ఒక 2-సభ్యుల group – ప్రత్యేక code అవసరం లేదు	0
"Simplify ని ఆపే ఎంపిక"	కొందరికి "నేను నేరుగా అతనికి ఇస్తాను" అని కావాలి	ఒక flag
"ఎవరైనా చెల్లించకపోతే?"	ఇది ఒక product సమస్య, technical కాదు – గుర్తుచేయడం, అంతే	

బహుళ currencies – కొంచెం లోతుగా

"ఒక్కో currency కీ ఒక్కో balance ఉంచుతాను – వాటిని కలపను. ఎందుకంటే మార్పిడి రేటు ఎప్పటిది అనే ప్రశ్న ఒక business నిర్ణయం: ఖర్చు జరిగిన రోజుదా, settle చేసే రోజుదా? రెండూ సమర్థనీయం, కానీ అవి వేర్వేరు మొత్తాలు ఇస్తాయి. నేను దాన్ని నేనే నిర్ణయించను – అడుగుతాను. అప్పటిదాకా currencies ని విడిగా ఉంచడం సురక్షితం."

17. ఏమి నేర్చుకున్నాం

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
సంఘటనలని store చెయ్యి, స్థితిని లెక్కించు	ఖర్చులు vs అప్పులు (§1)	Ledger, git, event sourcing, audit log
సరైన unit ఎంచుకుంటే సమస్య మాయమవుతుంది	జత → వ్యక్తి (§7)	ప్రతి modelling నిర్ణయంలో
Invariant ఒక self-check	మొత్తం ఎప్పుడూ సున్నా (§7)	Accounting, inventory, double-entry
Greedy మంచిది, కానీ కనిష్టం కాదు	Settlement (§9)	Scheduling, bin packing, coin change
పరిమితిని కొలవడం, ఊహించకపోవడం	86% కొలత (§10)	ప్రతి approximation lo
డబ్బుని పూర్ణాంకాల్లో	పైసలు (§12)	ఏ financial code lo అయినా

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot · 02 LRU & LFU Cache · 03 Rate Limiter · 04 BookMyShow · 05 Splitwise · 06 Elevator.**

ఈ ఐదు problems lo ఒక నమూనా: 01 మరియు 04 lo concurrency కష్టం. 02 మరియు 03 lo data structure కష్టం. ఇక్కడ – **modelling** కష్టం, మరియు code అతి చిన్నది. Interview lo మీ మొదటి పని ఏమిటంటే – **ఈ problem ఏ రకమో గుర్తించడం**, ఎందుకంటే దాన్ని బట్టి మీ సమయం ఎక్కడ పెట్టాలో తెలుస్తుంది.

వేగవంతమైన revision కోసం – [LLD_Design_Problems_Telugu.pdf](#) lo Problem 04. Strategy మరియు Decorator కోసం – [LLD_Telugu.pdf](#) §29, §18.

Splitwise – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output, మరియు 3,000-సమూహాల సరిపోలిక నిజంగా `node` lo run చేసి తీసినవే 